

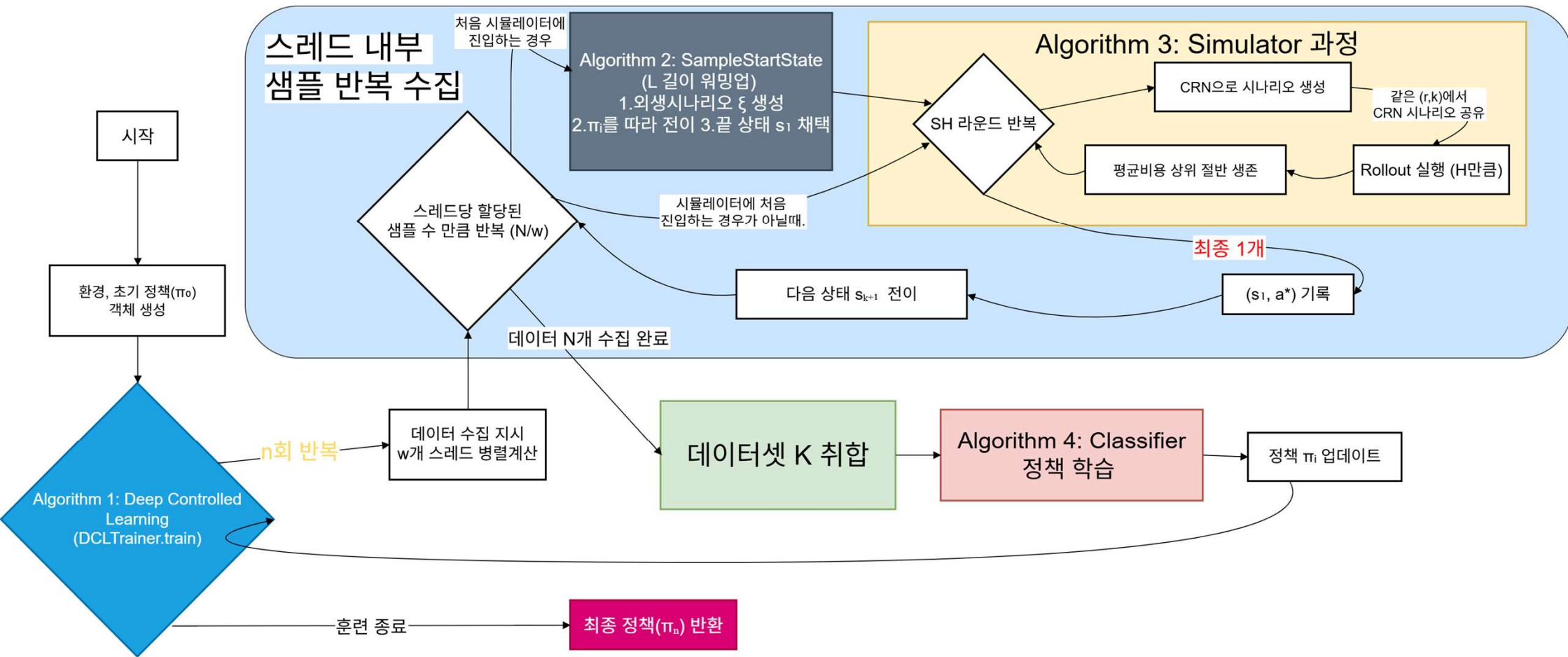
**DCL을 이용한
Lost sales / perishable inventory 정책개선 코드**

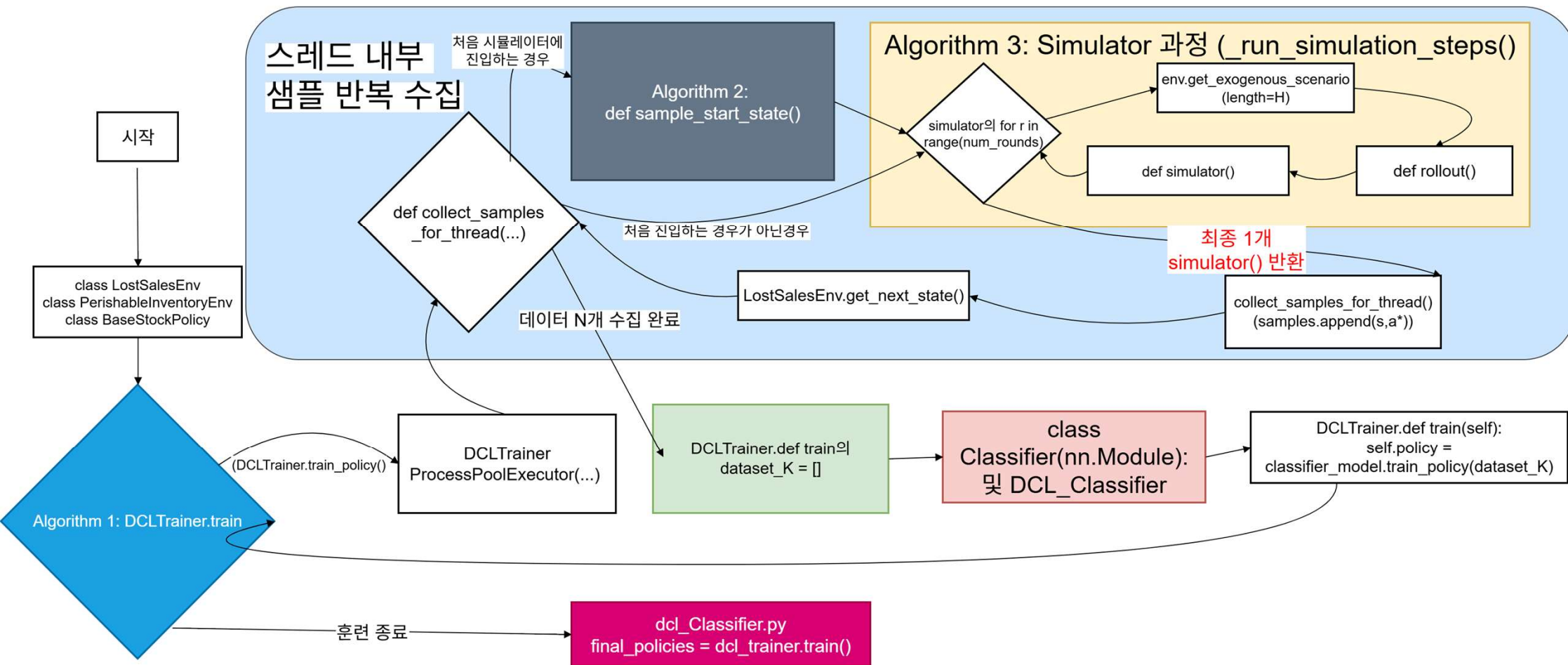
20181042 김경순



목차

- 1. 플로우차트를 통한 전체 코드 간략 설명
 - -초기 정책 및 환경
 - -DCL
 - -Sample Start State
 - -Simulator
 - -Classifier
- 2. 4가지 알고리즘 상세 설명





Part 1.

초기 정책 및 환경 함수

Class BaseStockPolicy : 초기 정책 설정.재고 정책의 기본 구조 (예: 주문 시점, 주문량 결정 규칙)

Class BaseInventoryEnv: 상태(s), 행동(a), 보상®, 전이함수를 관리하는 공통 구조

Class LostSalesEnv: 상태 전이와 비용 계산 로직 포함 (판매 손실 비용 반영)

Class PerishableInventoryEnv: 시간이 지나면 재고가 폐기되는 perish 로직 포함 (유통기한 재고)



Part 2.

Algorithm 1 - DCL

- **class DCLTrainer:**

DCL 전체 반복 클래스

[샘플 수집 - 학습 - 정책 개선]의
전체 흐름 담당

- **def**

collect_samples_for_thread

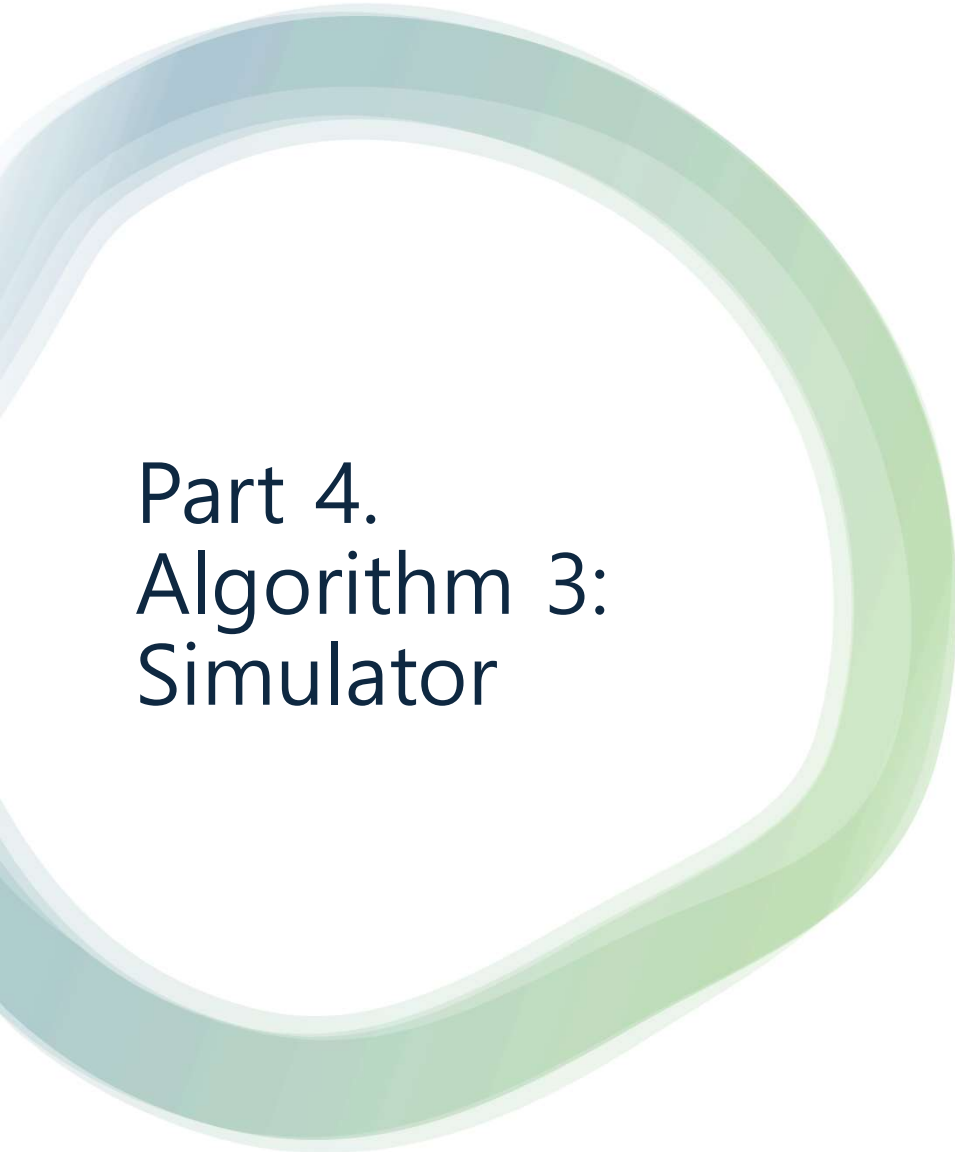
DCL 학습이 시작되면 N개의 샘플을
w개의 스레드를 통해 수집하도록 함

Part 3.

Algorithm 2 – Sample Start State

• **def sample_start_state**

- 개별 스레드에서 실행
- 각 병렬 스레드의 데이터 수집 시작점을 결정하는 함수
- 워밍업 과정을 통해 각 스레드는 다양한 상태에서 시작
- 다양한 상태를 반영하게 함으로써 견고한 정책개선에 도움
- `collect_samples_for_thread`에서 `current_state = sample_start_state(env, policy, L, start_day_index)` 를 통해 한번만 실행



Part 4. Algorithm 3: Simulator

- **def simulator:** 상태별 후보 평가 후 최적 행동 선택 함수
- **def rollout:** 정책 평가 실행 후 비용과 보상 평가
- **def _run_simulation_steps:** 행동 평가 반복 시행 (CRN 공유, halving 라운드 수행)
- **def collect_samples_for_thread**
 .local_samples.append
특정 상태 s 에 대해서 Simulator로 도출된
최적행동 a 를 리스트에 기록

Part 5.

Algorithm 4:

Classifier

- Classifier: 정책 근사 신경망, 수집된 데이터로 새로운 정책 학습
- `class Classifier(nn.Module):`
- 하이퍼 파라미터, 변수 받음
- Simulator로 생성된 데이터셋 K를 바탕으로 최선의 정책 $\pi(n)$ 를 학습을 시작.

1. DCL

- DCL Trainer : DCL 학습의 전체 과정을 n번 반복하며 총괄하는 메인 클래스
- collect
- 프로세스 –
 1. 데이터 생성 : 현재 가지고 있는 정책을 활용하여 대규모 데이터셋 K 생성(Sample Start State + Simulator)
 2. 정책 학습 : 생성된 데이터셋 K를 이용하여 신경망 분류기(Classifier)를 학습
 3. 정책 업데이트 : 학습이 완료된 신경망이 더 좋은 정책 사용
 4. 반복 학습 : 위의 학습을 n회만큼 실행

```
class DCLTrainer:
```

```
    """DCL 알고리즘의 전체 학습 과정을 관리합니다."""
```

```
    def __init__(self, env, initial_policy, n, N, M, H, L, w):
```

```
        self.env, self.policy = env, initial_policy
```

```
        self.n, self.N, self.M, self.H, self.L, self.w = n, N, M, H, L, w
```

```
    def train(self):
```

```
        trained_policies = []
```

```
        classifier_model = Classifier(self.env.state_size, self.env.num_actions)
```

```
        classifier_model.to(device)
```

```
        print(f"DCL Trainer가 {device} 장치를 사용하여 학습을 시작합니다.")
```

```
        for i in range(self.n):
```

```
            print(f"\n===== 시작: 근사 정책 반복 {i+1}/{self.n} =====")
```

```
            print(f"데이터셋 K_{i} 생성을 시작합니다 (총 {self.N}개 샘플, {self.w}개 워커)...")
```

```
            dataset_K = []
```

```
            samples_per_worker = math.ceil(self.N / self.w)
```

```
            # =====
```

```
            # <<k 수정된 부분 >>>
```

```
            # 리스트 컴프리헨션을 사용하여 각 워커에 대한 인자(args)를 생성합니다.
```

```
            # 이 루프가 돌 때마다 np.random.randint(0, 7)가 새로 호출되어
```

```
            # 각 워커는 서로 다른 무작위 시작 요일을 할당받게 됩니다.
```

```
            # =====
```

```
            args_list = [
```

```
                (self.env, self.policy, samples_per_worker, self.H, self.M, self.L, np.random.randint(0, 7))
```

```
                for _ in range(self.w)
```

```
            ]
```

1. 데이터의 개수 N개만큼
전체 학습과정 반복
2. N번만큼 전체학습 반복

```
# concurrent.futures를 import 해야 합니다.->병렬연산하기 위해  
from concurrent.futures import ProcessPoolExecutor, as_completed  
with ProcessPoolExecutor(max_workers=self.w) as executor:
```

2. 데이터셋 K_i 생성 시작(병렬처리)
W개의 스레드를 사용하여
collect_samples_for_thread 함수를 동시에 실행

```
    futures = {executor.submit(collect_samples_for_thread, args): i for i, args in enumerate(args_list)}  
    for future in as_completed(futures):
```

```
        try:
```

```
            dataset_K.extend(future.result())
```

병렬 실행 및 결과 취합

```
        except Exception as exc:
```

```
            print(f'워커에서 예외 발생: {exc}')
```

```
print(f"\n데이터셋 생성 완료. 총 {len(dataset_K)}개 샘플 수집.")
```

```
# --- 정책 학습 단계 ---
```

```
self.policy = classifier_model.train_policy(dataset_K)
```

3. 생성된 데이터셋으로 새로운 정책(신경망)을 학습

```
trained_policies.append(self.policy)
```

4. 학습된 정책을 저장 (선택적)

```
return trained_policies
```

```

def collect_samples_for_thread(args):
    env, policy, num_samples, H, M, L, start_day_index = args # <<< start_day_index 추가
    local_samples = []
    current_state = sample_start_state(env, policy, L, start_day_index)
    # <<< 시작 요일을 고려하여 시나리오 생성
    current_day_idx = np.argmax(current_state[-7:])
    exogenous_scenario = env.get_exogenous_scenario(length=num_samples, start_day_index=current_day_idx)

    for k in range(num_samples):
        best_action = simulator(env, current_state, policy, M, H)
        local_samples.append((current_state.tolist(), best_action))
        demand = exogenous_scenario[k]
        current_state = env.get_next_state(current_state, best_action, demand)
        if k % 10 == 0:
            print(".", end='', flush=True)
    return local_samples

```

DCLTrainer에게 호출되어 각 스레드별로 데이터를 모으기 시작.

2. Sample Start State (SSS)

Sample Start State : 데이터 수집을 시작할 '시작 상태' 를 찾는 역할

w값에 따라 각 스레드를 배정 및 스레드 단위 StartStartState 함수 호출



이 코드에서는 기본 재고 상태를 0으로 시작



현재 정책을 사용하여 L 기간 동안 가상으로 재고 관리 실행



L일이 지난 후의 최종 상태를 데이터 수집을 위한 현실적인 시작상태로 반환 (L = 월업기간)

알고리즘 2: SampleStartState

```
def sample_start_state(env, policy, L, start_day_index=0): # <<< start_day_index 추가
    state = env.get_initial_state(start_day_index)
    # <<< 시작 요일을 고려하여 시나리오 생성
    warmup_scenario = env.get_exogenous_scenario(length=L, start_day_index=start_day_index)
    for j in range(L):
        action = policy.get_action(state)
        demand = warmup_scenario[j]
        state = env.get_next_state(state, action, demand)
    return state
```

1. 환경의 초기 상태에서 시작, 재고0

2. L 길이만큼의 워밍업용 수요 시나리오 생성

3. L번의 시간 단계(time step) 동안 현재의 정책을 따라 워밍업 실행

4. 워밍업이 끝난 후의 최종 상태를 반환

3. Simulator

- Simulator : 주어진 상태에 대해 최적의 행동을 결정하는 역할
- 역할을 4부분으로 나눠서 함수를 할당
- 프로세스 –
 1. 모든 행동에 대해 적은 횟수의 시뮬레이션 실행 후
평균 비용 계산, 평균 비용이 높은 하위 50% 탈락
 2. 살아 남은 행동들을 대상으로 더 많은 시뮬레이션을 실행
하여 평균 비용 계산, 하위 50% 탈락.
 3. 하나의 행동이 남을 때 까지 이 과정을 반복

```

def simulator(env, state, policy, M, H):
    """가독성을 개선한 simulator 함수"""
    actions = list(range(env.num_actions))
    if len(actions) <= 1: return actions[0] if actions else 0

    total_budget = M * len(actions)
    num_rounds = math.ceil(math.log2(len(actions)))
    accumulated_costs = {a: 0.0 for a in actions}
    total_sims = {a: 0 for a in actions}

    # --- SH 라운드 루프 (전체 흐름을 관리) ---
    for r in range(num_rounds):
        sims_this_round = math.ceil(total_budget / (len(actions) * num_rounds))

        # 복잡한 부분을 도우미 함수 호출로 단순화
        round_costs = _run_simulation_steps(env, state, policy, H, actions, sims_this_round)

        # 결과 취합
        for action in actions:
            accumulated_costs[action] += round_costs[action]
            total_sims[action] += sims_this_round

        # 행동 제거 로직
        if len(actions) > 1:
            avg_costs = {a: accumulated_costs[a] / total_sims[a] for a in actions}
            num_to_keep = math.ceil(len(actions) / 2)
            actions = sorted(actions, key=lambda a: avg_costs[a])[:num_to_keep]

    return actions[0]

```

1. 평가할 모든 행동 리스트 생성

2. 총 시뮬레이션 예산과 라운드 수 계산

M: 행동 하나당 평균적으로 할당된 시뮬레이션 횟수

행동의 개수가 절반씩 줄어드므로 log2 만큼의 라운드가 필요

3. 비용과 시뮬레이션 횟수를 기록할 딕셔너리 초기화

이벤 라운드에 각 행동에 대해 몇 번의 시뮬레이션을 돌릴지 계산

보조함수 def _run_simulation_steps 호출하여 한 라운드 안의 스텝들 모두 실행 및 완료

1. 각 행동의 '현재까지의 평균 비용'을 계산

2. 살아남을 행동의 개수를 계산 (현재의 절반)

3. 평균 비용이 낮은 순서대로 행동들을 정렬, 상위 num_to_keep 개수만큼만 선택하여 다음 라운드로 진출

_run_simulation_steps 함수 내부

알고리즘 3: Simulator (SH + CRN)

```
def _run_simulation_steps(env, state, policy, H, actions, sims_this_round):
```

Sims_this_round가 t_r? 총 비용이랑 관련있는거?

하나의 SH 라운드에 필요한 모든 시뮬레이션 스텝을 실행하고,
각 행동에 대한 비용의 '합계'를 반환하는 도우미 함수.
(이 함수의 이름 앞 '_'는 '내부에서만 사용하는 도우미 함수'라는 관례적 표현입니다)

```
    round_costs = {a: 0.0 for a in actions}
```

```
    for _ in range(sims_this_round):
```

```
        scenario = env.get_exogenous_scenario(length=H)
```

```
        for action in actions:
```

```
            round_costs[action] += rollout(env, state, action, policy, H, scenario)
```

```
    return round_costs
```

1. 공통으로 사용할 미래 수요 시나리오 생성 (CRN)

2. 현재 살아남은 모든 행동들에 대해 반복

3. rollout 함수를 호출하여 H 기간 동안의 각 행동에 대한 총 비용을 계산하고 누적

Roll out 함수

Simulator의 보조 함수: 단일 롤아웃 실행

H: 롤아웃 깊이 (얼마나 먼 미래까지 볼 것인가)

```
def rollout(env, start_state, initial_action, policy, H, scenario):
```

policy: 첫 스텝 이후의 행동을 결정할 현재의 최선택(신경망)

```
    total_cost = 0
```

```
    state = start_state
```

```
    # t=0 (첫 스텝)
```

```
    demand = scenario[0] Simulator가 지정한 행동인 'initial_action' 을 실행
```

```
    total_cost += env.get_cost(state, initial_action, demand)
```

```
    state = env.get_next_state(state, initial_action, demand)
```

```
    # t=1 부터 H-1 까지 (get_next_state가 요일 업데이트를 자동으로 처리)
```

```
    for t in range(1, H): 현재 학습된 'policy'를 따라 자율적으로 행동
```

```
        action = policy.get_action(state) policy가 현재 상태(state)를 보고 다음 행동(action)을 결정
```

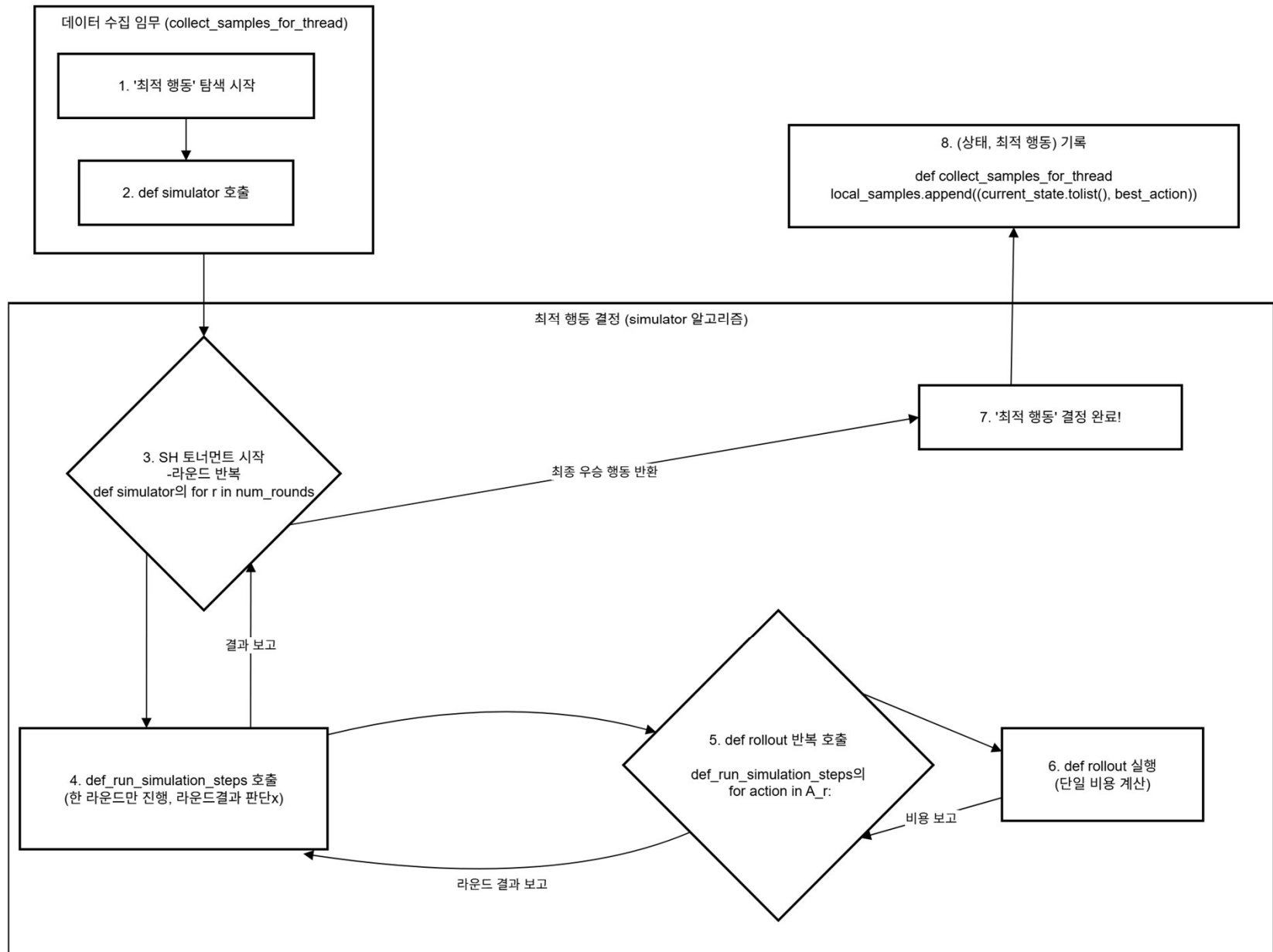
```
        demand = scenario[t]
```

```
        total_cost += env.get_cost(state, action, demand)
```

```
        state = env.get_next_state(state, action, demand)
```

```
    return total_cost
```

H 기간 동안의 각 상태, 행동, 수요에 따른 총 누적 비용을 total_cost에 반환,
run_simulation_step에서 쓰임



4. Classifier – 신경망부분

```
class Classifier(nn.Module):
```

```
    """
    상태(state)를 입력받아 최적 행동(action)을 예측하는 신경망 모델입니다.
    DCL의 정책 근사(Policy Approximation)를 담당합니다.
    """
```

```
    def __init__(self, input_size, num_actions):
        super(Classifier, self).__init__()
        # 논문에 제시된 신경망 구조 예시를 따름 (4-layer MLP)
        self.network = nn.Sequential(
            nn.Linear(input_size, 256),
            nn.ReLU(), # ReLU 활성화 함수
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Linear(128, 128),
            nn.ReLU(),
            nn.Linear(128, num_actions) # 최종 출력은 각 행동에 대한 점수(logit)
        )

    def forward(self, state):
        # 상태 텐서를 입력받아 각 행동에 대한 로짓을 반환
        return self.network(state)

    def get_action(self, state):
        """학습된 모델을 정책으로 사용할 때 호출되는 함수"""
        if isinstance(state, np.ndarray):
            # <--- GPU 수정 2: 상태 데이터를 device로 보냄
            state = torch.FloatTensor(state).unsqueeze(0).to(device)
        with torch.no_grad():
            logits = self.forward(state)
            action = torch.argmax(logits, dim=1).item()
        return action
```

가장 기본적인 형태의 딥러닝 모델인 MLP
활성화 함수 ReLU 채용

입력층: input_size 만큼의 노드(상태 벡터의 차원)를 가짐
은닉층 1: 256개의 뉴런.
은닉층 2 : 128개의 뉴런.
은닉층 3: 128개의 뉴런.
출력층: num_actions 만큼의 노드를 가짐
마지막 출력층은 각 행동이 얼마나 좋은지를 나타내는
원시 점수(logit)를 출력함.

forward(self, state)에서 순전파 진행.

get_action(self, state) 특정 state 정보를 입력받았을 때,
forward를 통해 각 행동의 점수를 계산하고,
torch.argmax를 사용해 그중 가장 높은 점수를 받은 행동을
'최적 행동'으로 결정하여 반환합니다.

4. Classifier – 신경망 부분

```
def train_policy(self, dataset_K, epochs=50, batch_size=64, learning_rate=1e-3):
```

```
    """
```

```
    알고리즘 4(딥러닝)의 학습 과정을 구현합니다.  
    (상태, 최적 행동) 데이터셋 k를 이용해 신경망을 학습시킵니다.
```

```
    """
```

```
    # 1. 데이터 준비: 데이터셋을 텐서로 변환하고 DataLoader 생성
```

```
    states, actions = zip(*dataset_K)
```

```
    states_tensor = torch.FloatTensor(np.array(states))
```

```
    actions_tensor = torch.LongTensor(np.array(actions))
```

```
    if len(states_tensor) < 5:
```

```
        print("데이터셋이 너무 작아 학습을 건너뛰니다.")
```

```
        return self
```

```
    # 훈련/검증 데이터셋 분할 (예: 80/20)
```

```
    dataset_size = len(states_tensor)
```

```
    val_size = max(1, int(0.2 * dataset_size))
```

```
    train_size = dataset_size - val_size
```

```
    full_dataset = TensorDataset(states_tensor, actions_tensor)
```

```
    train_dataset, val_dataset = torch.utils.data.random_split(full_dataset, [train_size, val_size])
```

```
    train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
```

```
    val_loader = DataLoader(val_dataset, batch_size=batch_size)
```

train_policy(self, dataset_K, ...)

가장 핵심적인 부분

데이터셋을 훈련/검증 셋으로 나눔

코드는 8:2로 나눈 모습

DCL중 Classifier 부분의 하이퍼파라미터를 볼 수 있음.

데이터셋이 너무 작으면 학습이 불가능하게 설계

Epoch=50

Batch_size=64

학습률=1e-3

4. Classifier – 신경망 부분

```
# 2. 옵티마이저 및 손실 함수 정의
optimizer = optim.Adam(self.parameters(), lr=learning_rate) # 옵티마이저로 가장 보편적인 아담 사용
loss_fn = nn.CrossEntropyLoss() # 분류 문제의 표준 손실 함수

best_val_loss, patience_counter, patience = float('inf'), 0, 5 # 조기 종료(Early Stopping)를 위한 대기 횟수
print("신경망 학습 시작...")
# 3. 학습 루프
for epoch in range(epochs):
    self.train() # 모델을 훈련 모드로 설정
    for batch_states, batch_actions in train_loader:
        # <--- GPU 수정 3: 훈련 데이터 배치를 device로 보냄
        batch_states = batch_states.to(device)
        batch_actions = batch_actions.to(device)
        # 순전파
        outputs = self.forward(batch_states)
        loss = loss_fn(outputs, batch_actions)

        # 역전파 및 파라미터 업데이트
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()]
```

옵티마이저: Adam을 사용

검증 및 조기 종료(Early Stopping): 만약 일정 횟수 (patience) 동안 손실값이 더 이상 좋아지지 않으면, 학습이 충분히 되었거나 과적합의 위험이 있다고 판단하여 학습을 미리 중단한다.

손실함수로 CrossEntropyLoss를 사용

$$-l(\mathbf{s}, \hat{\pi}^+(\mathbf{s})|\theta) := -\log \frac{\exp(N_{\theta}(\mathbf{S})[\hat{\pi}^+(\mathbf{S})])}{\sum_{a' \in \mathcal{A}_{\mathbf{S}}} \exp(N_{\theta}(\mathbf{S})[a'])}$$

4. Classifier – 신경망 부분

```
# 4. 검증 및 조기 종료
self.eval() # 모델을 평가 모드로 설정
current_val_loss = 0
with torch.no_grad():
    for batch_states, batch_actions in val_loader:
        # <--- GPU 수정 4: 검증 데이터 배치를 device로 보낼
        batch_states = batch_states.to(device)
        batch_actions = batch_actions.to(device)
        outputs = self.forward(batch_states)
        loss = loss_fn(outputs, batch_actions)
        current_val_loss += loss.item()

avg_val_loss = current_val_loss / len(val_loader) if val_loader else 0
print(f"Epoch {epoch+1}/{epochs}, Validation Loss: {avg_val_loss:.4f}")

if avg_val_loss < best_val_loss:
    best_val_loss, patience_counter = avg_val_loss, 0
else:
    patience_counter += 1
if patience_counter >= patience:
    print("조기 종료(Early stopping) 발동!")
    break
print("신경망 학습 완료.")
return self # 학습된 모델 반환
```

for ... in val_loader 부분에서 검증셋 데이터를 불러옴
검증셋에 대해 최종 정책을 적용하고 손실을 계산.
current_val_loss에 계속 loss를 더해감.

avg_val_loss(이번 검증의 평균 loss)와
best_val_loss(역대 제일 낮은
검증 loss 기록)을 비교해 갱신하거나 하지않음.

갱신하면 patience를 초기화
갱신하지못하면 patience_counter를 1늘림.

지정한 patience값(Early Stopping 값)에 다다르면
조기종료

4. Classifier – 파라미터 조정 및 실행

```
import torch
import numpy as np
import math
from concurrent.futures import ProcessPoolExecutor, as_completed
import datetime
```

셀 1: Lost Sales 문제

```
from dcl_definitions_refactored import (
    LostSalesEnv,
    BaseStockPolicy,
    DCLTrainer,
    collect_samples_for_thread
)
```

셀 2: Perishable Inventory 문제

```
from dcl_definitions_refactored import (
    PerishableInventoryEnv,
    BaseStockPolicy,
    DCLTrainer,
    collect_samples_for_thread
)
```

필요한 라이브러리, 모듈 등을 불러옴.
기존 dcl_definitions.py에서 작성된 대부분의 알고리즘을
위해 필수.

4. Classifier – Lost sales

```
# --- 1. 문제 및 하이퍼파라미터 정의 ---
print("="*50)
print("Lost Sales 문제에 대한 DCL 학습을 시작합니다.")
print("="*50)

# 변수 이름이 겹치지 않도록 '_ls' 접미사를 사용합니다.
env_ls = LostSalesEnv(
    lead_time=3,
    holding_cost=1000,
    penalty_cost=500,
    mean_demand=13,
    max_action=35
)
initial_policy_ls = BaseStockPolicy(base_stock_level=35)#초기정책(=BSP)을 1-1로 구현 35로 설정

# DCL 하이퍼파라미터
n = 2      # 정책 반복 횟수
N = 300    # 총 데이터 샘플 수, 오래걸려서 500->300으로 낮춤
M = 50     # 상태-행동 쌍 당 외생 시나리오 수, 총 시뮬레이션 예산(B_s)을 결정하는 값, 오래걸려서 100에서 50으로 낮춤
H = 20     # 돌아옴 깊이
L = 20     # 월업 기간, 초기값 50->20으로 낮춤
w = 6      # 병렬 워커(프로세스) 수, w는 작업 프로세스에서 CPU 논리프로세스 수에 따라서
```

각 문제에 맞는 환경을 불러오고,

환경에 대한 파라미터

알고리즘에 대한 하이퍼파라미터를 입력

수요의 전체 평균과 표준편차를 받아서 정규분포로 요일별 '평균 수요값'을 생성

외생 시나리오 생성 시 '평균 수요값' = λ 를 만족하는 포아송분포를 통해 생성

4. Classifier – Lost sales

```
# --- 2. DCL 트레이너 생성 및 학습 실행 ---
dcl_trainer_ls = DCLTrainer(env_ls, initial_policy_ls, n, N, M, H, L, w)
final_policies_ls = dcl_trainer_ls.train()

# --- 3. 결과 ---
best_policy_lost_sales = None
if final_policies_ls:
    best_policy_lost_sales = final_policies_ls[-1]
    print("\n학습 완료! 'best_policy_lost_sales'가 생성되었습니다.")

    example_state_ls = np.array([5, 10, 8]) # 벡터 차원 = 리드타임 ( $\tau$ ) 이므로, Lead_time의 값에 따라서 벡터의 차원을 맞춰줘야함.
    action_ls = best_policy_lost_sales.get_action(example_state_ls)
    print(f"상태 {example_state_ls}에서 학습된 정책의 결정 행동: {action_ls}")
else:
    print("\nLost Sales 학습이 완료되지 않았습니다.")
```

dcl_trainer = DCLTrainer(...) (위의 경우에선 dcl_trainer_ls)
부분에서 각 파라미터를 반영하여 DCLTrainer기반의 객체를 생성.

final_policies_ls = dcl_trainer_ls.train()
훈련을 시작하라는 코드
best_policy_lost_sales = final_policies_ls[-1]
가장 최근의 업데이트된, 성능이 제일 좋은 정책을 저장

=====
Lost Sales 문제에 대한 DCL 학습을 시작합니다.
=====

=====
Lost Sales: 자동 생성된 요일별 평균 수요:
- 월: 158.64
- 화: 191.03
- 수: 164.97
- 목: 169.64
- 금: 62.06
- 토: 202.69
- 일: 200.59
=====

DCL Trainer가 cuda 장치를 사용하여 학습을 시작합니다.

===== 시작: 근사 정책 반복 1/2 =====
데이터셋 k_0 생성을 시작합니다 (총 300개 샘플, 6개 워커)...

데이터셋 생성 완료. 총 300개 샘플 수집.
신경망 학습 시작...

Epoch 1/50, Validation Loss: 4.5802
Epoch 2/50, Validation Loss: 4.2181
Epoch 3/50, Validation Loss: 4.0242
Epoch 4/50, Validation Loss: 4.0364
Epoch 5/50, Validation Loss: 4.0437
Epoch 6/50, Validation Loss: 4.0610
Epoch 7/50, Validation Loss: 4.0903
Epoch 8/50, Validation Loss: 4.1496
조기 종료(Early stopping) 발동!
신경망 학습 완료.

===== 시작: 근사 정책 반복 2/2 =====
데이터셋 k_1 생성을 시작합니다 (총 300개 샘플, 6개 워커)...

데이터셋 생성 완료. 총 300개 샘플 수집.
신경망 학습 시작...

Epoch 1/50, Validation Loss: 7.6833
Epoch 2/50, Validation Loss: 6.4855
Epoch 3/50, Validation Loss: 6.0692
Epoch 4/50, Validation Loss: 5.8417
Epoch 5/50, Validation Loss: 5.7237
Epoch 6/50, Validation Loss: 5.7339
Epoch 7/50, Validation Loss: 5.7224
Epoch 8/50, Validation Loss: 5.8542
Epoch 9/50, Validation Loss: 6.0861
Epoch 10/50, Validation Loss: 6.2352
Epoch 11/50, Validation Loss: 6.3118
Epoch 12/50, Validation Loss: 6.3965
조기 종료(Early stopping) 발동!
신경망 학습 완료.

학습 완료! 'best_policy_lost_sales'가 생성되었습니다.
상태 [5 10 8 0 0 0 0 1 0 0]에서 학습된 정책의 결정 행동: 180

1. 환경 파라미터 (문제 정의)

- 리드타임 (lead_time): 3일
•재고 유지비 (holding_cost): 1000
•재고 부족 페널티 (penalty_cost): 5000
•전체 평균 수요 (mean_demand): 150
•요일별 수요 편차 (demand_std_dev): 50
•최대 주문량 (max_action): 300
•초기 정책 목표 수준 (base_stock_level): 400

2. DCL 하이퍼파라미터 (학습 방법)

- 정책 반복 횟수 (n): 2회
•데이터 샘플 수 (N): 300개
•시뮬레이션 예산 (M): 10
•롤아웃 깊이 (H): 20 기간
•웍업 기간 (L): 10 기간
•병렬 워커 수 (w): 6개

4. Classifier – Perishable Inventory

```
# --- 1. 문제 및 하이퍼파라미터 정의 ---
```

```
print("\n" + "="*50)
```

```
print("Perishable Inventory 문제에 대한 DCL 학습을 시작합니다.")
```

```
print("="*50)
```

```
# 변수 이름이 겹치지 않도록 '_pi' 접미사를 사용합니다.
```

```
env_pi = PerishableInventoryEnv(
```

```
    lifetime=3,
```

```
    lead_time=2,
```

```
    holding_cost=1,
```

```
    penalty_cost=100,
```

```
    waste_cost=2,
```

```
    mean_demand=15, # 전체 요일의 평균 수요는 15
```

```
    demand_std_dev=5, # 요일별 평균 수요의 표준편차는 5
```

```
    max_action=40,
```

```
    fifo_ratio=0.5
```

```
)
```

```
initial_policy_pi = BaseStockPolicy(base_stock_level=60)#초기정책인 BSP를 60으로 설정
```

```
# DCL 하이퍼파라미터
```

```
n = 2 # 정책 반복 횟수
```

```
N = 300 # 총 데이터 샘플 수, 오래걸려서 500->300으로 낮춤
```

```
M = 50 # 상태-행동 쌍 당 외생 시나리오 수, 총 시뮬레이션 예산(B_s)을 결정하는 값, 오래걸려서 100에서 50으로 낮춤
```

```
H = 20 # 롤아웃 깊이
```

```
L = 20 # 율업 기간, 초기값 50->20으로 낮춤
```

```
w = 6 # 병렬 워커(프로세스) 수, w는 작업 프로세스에서 CPU 논리프로세스 수에 따라서
```

Definitions.py에서 문제에 따른 환경클래스를 불러오고 초기 환경을 세팅.

DCL 하이퍼파라미터를 받아서 학습할 준비

4. Classifier – Perishable Inventory

```
# --- 3. 결과 ---
best_policy_perishable = None
if final_policies_pi:
    best_policy_perishable = final_policies_pi[-1]

# <<< 테스트 상태에 요일 정보 추가 >>>
# 예시 1: 금요일(인덱스 4)이고, 재고가 없는 상황
inventory_part = np.zeros(env_pi.m1 + max(env_pi.tau - 1, 0))
day_part_fri = np.array([0, 0, 0, 0, 1, 0, 0])
friday_state = np.concatenate((inventory_part, day_part_fri))

action_fri = best_policy_perishable.get_action(friday_state)
print(f"금요일에 재고가 없을 때 추천 주문량: {action_fri}")
```